

Lecture 1

Sunday, 22 February 2026 1:22 am

- Agent is something that observes its environment (using sensors) and acts on the environment (using effectors)
- Agentic AI is AI that makes decisions on its own, has goals and acts independently

Passive vs active agents

- Passive agents
 - React only to current input
 - Have no memory
 - No learning
 - No planning
 - e.g. motion sensor light -> detects movement -> turns on
- Active agents
 - Have memory
 - Learn from past
 - Plan actions
 - Work toward goals
 - e.g. GPS -> remembers ur destination -> calculates best route

Reactive vs deliberative agents vs proactive agents

- Reactive agents
 - Immediate response
 - No thinking ahead
 - If condition
 - Rule-based
 - e.g. car auto brake system
- Deliberative agents
 - Think before acting
 - Make plans
 - Consider goals
 - Strategic thinking
 - e.g. chess playing AI
- Proactive agents
 - Does not wait
 - Takes initiative based on internal goals
 - Acts before problems happen
 - e.g. antivirus software updating itself, checks for updates regularly

Software vs embodied agents

- Software agents
 - Exist in computer
 - Work in virtual environments
 - Sensors are APIs, user input and effectors are sending emails, writing files
 - Focus on thinking, reasoning, processing information
 - e.g. chatbot
- Embodied agents
 - Physical body, can interact with real world
 - Sensors are camera, microphones and effectors are motors, wheels, arms
 - Focus on physical movement, navigation
 - e.g. robot

Agentic AI

- It sets goals, plans actions, learns, adapt, works independently
- An AI is agentic if it has
 - Autonomy: acts without constant instructions
 - Goal-directed behaviour: works towards objectives
 - Memory: remembers past
 - Decision authority: decides what to do next
- A system can be smart but not agentic
 - Single gpt prompt only responds when asked but do not decide what to do next
- LLMs as reasoning agents
 - They understand instructions, create plans and use tools but they may give wrong information, they have no real world experience and can be biased
- Human control levels
 - Human in the loop
 - Human must approve
 - Used in finance and healthcare
 - Human on the loop
 - AI acts independently and human monitors
 - Fully autonomous
 - Minimal human control and high risk

Agentic AI reasoning frameworks

- ReAct (reason + act)
 - Think -> act -> reflect -> repeat
 - Flexible and adaptive
 - External tools
- Reflexion
 - Agent acts, reviews its own mistake and tries again
- Plans and executes
 - Step1: make full plan
 - Step2: execute steps
 - Organized execution but not flexible
- Tree of thought
 - Agent explores multiple options
- Chain of thought
 - Step by step reasoning
- Multi agent systems
 - Multiple agents work together
- LLMs + tools
 - Agent can use APIs, use search engines, use calculator

Agentic AI development frameworks

- LangChain: tools +memory
- AutoGen: multi agent collaboration
- crewAI: role based agents
- LlamaIndex: knowledge retrieval
- BabyAGI: simple autonomous agents
- SuperAGI: large scale agents

Reasoning framework: how the agent thinks

Development framework: where and how you build it

Lecture 2

Sunday, 22 February 2026 2:03 am

Prompt engineering

- Technique of designing clear and structured instructions for LLM
- It converts raw models into structured reasoning agents, improves reliability and accuracy, acts like a user interface to AI reasoning

Zero shot prompting

- No example given
- Model relies on pretraining knowledge
- Works well for simple, common facts
- If asked about unknown events, they may invent answers confidently as LLMs don't know the truth

Few shot prompting

- Provide 2-5 examples before asking real questions
- Best for classification, structured output, extraction
- If examples are wrong, model copies mistakes

Chain of thought

- Ask model to reason step by step before answering
- Improves math, logic and planning
- Longer reasoning=more chance of error
- Model may fill gaps with fabricated facts when token length exceeds

Tree of thought

- Explore multiple reasoning paths before choosing best

Role and system prompting

- System prompt sets global rule (tone, ethics)
- Role prompt defines job of the model

Multi context prompting

- Divide input into layers
- Context A: knowledge
- Context B: constraints
- Context C: output style
- It reduces hallucination as it organizes reasoning, makes output traceable, works well in multi agent systems

AutoPrompt

- Prompts are automatically discovered using optimization
- Algorithms search tokens that maximizes performance

ReAct prompting

- Agent calls tools during reasoning
- It prevents hallucinations as it gives pure reasoning but might guess numbers
- It uses real tools and each observation step anchors reasoning

Least to most prompting

- Break complex problem into smaller ones and solve easiest first and reduces cognitive load

Self-ask prompting

- Model generates its own sub questions and prevents skipping reasoning steps

Meta prompting

- Prompt that creates another prompt, reduces manual effort
- Can become vague or too generic

Symbolic reasoning prompting

- Encourage formal logic representation, reduces ambiguity

Program aided language

- LLM writes code to solve problem
- More reliable for maths

Iterative prompting

- Ask -> review -> refine -> improve
- Feedback loop
- Output becomes better at each round

Sequential prompting

- Chain prompts, each response becomes input to the next step

Self-consistency prompting

- Generate multiple reasoning paths
- Choose majority answer, reduces random errors

Generated knowledge prompting

- Generate background facts, use those facts to answer

To reduce hallucinations

- Ask for sources
- Use self consistency
- Apply iterative prompting
- Combine with RAG
- Use tool calling ReAct

Lecture 3: langchain

Sunday, 22 February 2026 2:05 pm

Why use a framework?

- Reduces boilerplate
 - Without langchain we write loops, parse text, handle errors manually but with langchain loop handled automatically and cleaner code
- Standard tool interface
 - Without langchain each tool has custom format but with langchain every tool has name, description, input schema and output schema
 - Helps llm understand how to call tools directly
- Observability and debugging
 - Without langchain we print logs manually but with langchain it shows reasoning trace, shows tool calls and output hence easier debugging
- Plug and play ecosystem
 - Langchain provides integrations for search engines, vector databases, SQL databases and APIs

What is LangChain?

- A workflow framework, toolkit, an organizer for LLM applications
- It has 2 core ideas
 - Chains: predefined steps, always the same flow
 - Agents: decides which tool to use, when to use it and how many times
- Without langchain we build reasoning loop ourselves but with langchain we only define tools, provide prompts and choose output format
- It handles memory, tool calls, execution loop and supports reasoning strategies such as ReAct, plan-execute and self ask

Chains vs agents

- Chains are predictable, fast, reliable, easy testing; always the same steps
- Agents are flexible, dynamic tool selection, real world adaptable
- When to use what

Manual ReAct vs LangChain agent

- Manual react
 - We wrote parsing, handled it manually and tools were plain python functions
- Langchain agent
 - Built in ReAct prompt, built in loop manager, tool wrapper format, agentexecutor handles reasoning cycle

Agent building blocks

- LLM: reasoning engine
- Tools: external functions or APIs
- Prompt template: guides reasoning (thought ->Action ->observation)
- AgentExecutor: coordinates with LLM and tools; runs loop until final answer

What counts as a tool?

- Anything that an agent can call
- Knowledge tools: wikipedia search, web search
- Utility tools: calculator
- Data tools: SQL database, API returning JSON
- Enterprise connectors: slack, gmail

Safety and guardrails in LangChain

- LangChain provides
 - Step limits: limit number of tool calls
 - Tool input validation: tools required structured schema

- Restricted tools: sensitive tools can be disabled
 - Moderation&filtering: block unsafe output
- When not to use an agent?
 - Summarizing tasks; use chains instead
 - When no tools are required; use LLM
 - Latency sensitive: use LLM
 - Stability > flexibility: agents choose path dynamically; use chains if deterministic output required

Lecture: llamaindex vs langchain

Sunday, 7 June 2026 2:21 pm

RAG

- Where external documents are retrieved first, the supplied to the LLM as context and only the LLM generates an answer
- Elements of RAG
 - Retrieval: finding relevant documents or chunks from an external knowledge base
 - Augmentation: taking those retrieved chunks and adding them to the prompt as context
 - Grounded generation with citations: LLM generates answer based on retrieved content

Why not rely only on LangChain?

- It is good at
 - Orchestration: chaining multiple prompts together in a sequence
 - Managing agents: deciding which tool to use and in what order
 - Tool use: connecting to APIs, calculators, search engines
- It struggles with
 - Rich indexing strategies: it can store documents in a vector database, but doesn't have advanced options for how to organize and index that data
 - Document observability: limited debugging and transparency so hard to know that why a specific chunk was retrieved
 - Complex retrieval graphs: for query across multiple document, its retrieval setup becomes very limited

Why llamaIndex

- Ingesting data, indexing it intelligently and retrieving it efficiently
 - Index types
 - Vector index: uses embeddings for similarity based search
 - Tree index: organizes documents in a hierarchical tree for summarization
 - Keyword table: fast lookups based on keywords, useful for structured documents
 - List index: for linear sequential documents where order matters
- Query engines: handles retrieval, reranks results by relevance and synthesizes the final answer
- Observability: inspect exactly which nodes were retrieved and track where they came from

Benefits of llamaIndex for RAG

- Strong data connectors: can connect to wide variety of sources: databases, pdf, word files, web pages etc
- Multiple retrieval strategies: dense retrieval (embedding-based semantic search), sparse retrieval (exact keyword matching), hybrid, reranking (reorders the results to put the most relevant chunks first)
- Composable retrieval graphs: can query across multiple document at once and combine the results
- Observability tools: easy debugging as we can see why a chunk was retrieved and what similarity score it had

llamaIndex limitations

- Retrieval adds extra latency as it searches through the vector databases
- Index maintenance can be costly as documents grow, it needs to re-embed and re-index them
- Less flexible for arbitrary tool use as it focuses on heavily on retrieval and not for handling general tool orchestration

Langchain + llamaIndex

- Langchain handles the orchestration, agent reasoning, tool coordination and decision making
- llamaIndex is the memory. It handles indexing, retrieval and document observability
- Limitation combined
 - Integration complexity

- Making them work together requires significant engineering effort as they were built by different teams with different APIs
- Higher resource
 - More memory, more compute and higher costs
- Monitoring must span across both systems if something goes wrong

Vector databases

- It stores embeddings- numerical vector representation of text
- The query is also converted to a vector and the database finds the stored vectors closest it in meaning e.g. chroma, FAISS
- Can search millions of vectors quickly, can filter by document type, embeddings are saved on disk and don't need to be recreated every time

Memory in RAG systems

- During a conversation, retrieved chunks can be stored in local memory so the system remembers what was retrieved earlier in the conversation
- Short-term memory
 - Local-session memory that keeps context across multiple turns of a conversation. It resets when the session ends
- Long-term memory (vector db)
 - Persists permanently and holds the full document knowledge base

RAG pipeline

- Ingestion phase
 - Loaders: bring in raw documents
 - Text splitters: break large documents into smaller, manageable chunks
 - Embeddings: convert each chunk into numerical vector
 - Vector DB: store all the embeddings
- Runtime phase
 - User query
 - Retriever: query is embedded and top k most similar chunks are fetched from vector db
 - LLM: retrieved chunks are added to the prompt as context and Llm generates answer
 - Response

When to use RAG

- Knowledge changes frequently
- Citations are needed
- Private/internal data must be accessed
- Reducing hallucination risk is critical

Limitations of RAG

- Retrieval quality depends on embeddings so if the embedding model doesn't capture meaning well then the wrong chunk will be retrieved
- Chunk size trade off
 - If it is too big then chunk contains too much irrelevant information
 - If it is too small then chunk loses surrounding context and doesn't make sense on its own
- Retrieval errors because LLM can hallucinate if relevant chunk is not retrieved
- Latency added as every query requires a round trip to the vector db

How to decide chunk size

- Start from embedding models context window
- Aim for standalone meaning as the chunk should make sense to a human on its own
- Precision vs generality trade off
 - Smaller chunks= more precise embeddings
 - Larger chunks= more general embeddings
- Llamaindex default
- Adjust retrieval depth when changing chunk size

Lecture 4: memory in agentic AI

Tuesday, 14 April 2026 5:09 pm

What is memory in AI?

- It means to store information, recall it later and use it to influence decisions
- It is not just a database or chat history

4 layers of memory in LLM systems

- Working memory (inside model)
 - Memory inside the LLM
 - It exists only while processing input
 - To compute calculations, it holds number temporarily and computes result then immediately forgets after response
 - Context window is amount of text LLM can see at once
 - If context window token is 8000 tokens, then anything other than that is forgotten by LLM, oldest tokens are removed
 - How attention acts as memory
 - It helps model focus on important words and connects relationships but only within the context window
 - It is not long term memory, so when conversation ends the model forgets
 - During generation, model stores key-value pairs to speed up computation and avoid recomputing tokens, used for optimization
 - Limitations of working memory
 - Token limit
 - Recent data dominates
 - No past session memory
 - Cannot store large data
 - Cannot search old information
- Session memory (application level)
 - The memory is not inside the model
 - It is managed by frameworks like langchain
 - It first stores conversation externally and then injects prompt when needed e.g. a chatbot that remembers ur name and preferences as it stores it externally
 - ConversationBufferMemory stores full conversation history but uses many tokens and is expensive
 - ConversationSummaryMemory stores summarized old messages instead of everything, it saves tokens and keeps context but can lose important details
 - TokenBufferMemory stores only last N tokens to prevent overflow, but still temporary and no long term memory
 - Where do buffers sit?
 - User > memory buffer > LLM
 - Memory is an external system and LLM is a stateless engine
 - Session memory is limited and agentic systems need persistent memory, cross session memory, document retrieval, shared memory across agents and task history
- What is external memory
 - Any storage that is outside the LLM, includes vector databases, SQL databases
 - LLMs need external memory since they only have limited context window and no learning across sessions
 - Risk
 - Wrong/malicious info enters memory as LLM trusts retrieved data blindly so verify

- document first or use trusted sources
 - Same document is retrieved repeatedly because of similar embedding so same answers, no diversity
 - Important info split across chunks so meaning is lost
 - Too many documents are retrieved
- **What is a vector database**
 - It stores meaning and not just text
 - Process flow
 - Text is converted into vector embeddings
 - Each entry contains vector (number representing meaning), original text, metadata
 - Semantic search allows to search by meaning not keywords
 - How vector memory works
 - Converts documents -> embeddings, then store it in vector DB, then convert query -> embeddings, compare similarity, lastly retrieve top K results
 - Retrieval is memory recall
 - What is stored in vector db
 - Each record contains embedding vector, text chunk, metadata
- **Semantic vs episodic memory**
 - Semantic memory are facts: knowledge, discovery, RAG content
 - Episodic memory are from past experiences, failures and task history
 - Agent requires both
 - Semantic pipeline
 - User -> planner -> retriever -> evidence -> LLM
 - Retriever is the gateway to memory and vector db is long term semantic memory
 - Episodic pipeline
 - Planner -> retrieve -> critic -> refine
 - It stores past queries, agent decisions, success and failure logs
- **Agent memory loop**
 - Query -> retrieve -> evaluate -> store outcome -> adjust
 - These are self improving agents
- **Hybrid memory architecture**
 - Working memory -> context window
 - Session memory -> chat buffer
 - Vector semantic memory -> knowledge
 - Episodic memory -> experience
- **Design decisions in memory systems**
 - What to store, how long to store, how to retrieve and how to rank
- **Memory in agentic RAG system**
 - Planner -> retriever -> critic -> refine -> final
 - Hierarchy
 - Context window = working memory
 - Buffer = session memory
 - Vector db = semantic memory
 - Logs = episodic memory

Lecture 5: MultiAgent systems

Tuesday, 14 April 2026 7:55 pm

- Single agents are overloaded, confused by complex workflows, and hard to scale
- Even a single agent with many roles fail because of
 - Context overload: too much instruction at once
 - No true separation of tasks: roles are just text
 - No verification step: doesn't check its own work properly
 - No state tracking
- **What a multi agent system would do?**
 - Multiple agents have different roles
 - Agents communicate via structured messages
 - They have shared state and each step is validated before moving forward
 - Features
 - Decomposition and specialization
 - It breaks big tasks into smaller ones
 - Coordination and orchestration
 - Agents communicate and pass work until previous one validates it
 - Scalability and parallelism
 - Multiple agents can run together e.g. 3 agents scraping data simultaneously
 - Robustness and fault tolerance
 - If one fails others can fix
 - Dynamic adaptability
 - System can change during execution
 - Clear responsibility
 - Each agent has defined job
 - Core principles
 - Autonomy
 - Each agent acts independently
 - proactivity
 - Agents align their work
 - Social ability
 - Agents exchange message
 - reactivity
 - Agents adjust behaviour

CORE BUILDING BLOCKS

- **Autonomous agents collaboration types**
 - Cooperative - teams work together
 - Competitive
 - Mixed
- **Messaging protocols**
 - Agents must exchange info clearly
 - Components include sender, receiver, message type, content, timestamp
 - Communication types include
 - FIPA-ACL
 - Standard language for agents
 - Asynchronous
 - Non blocking e.g. whatsapp
 - Send message and continues to work
 - Synchronous

- An agent keeps on waiting until other agent replies so blocking
- Best practices
 - Clear message schema
 - Acknowledgement of message received
 - Retry mechanism if message fails
 - Avoid repeating same messages
- **Coordination strategies**
 - Combination of coordination, communication and collaboration
 - Centralized
 - One controller assigns tasks to subagents
 - Hierarchical
 - Supervisor then sub team then workers
 - Decentralized
 - Agents coordinate via shared signals
 - Market based
 - Tasks are auctioned, best bidder executes
 - Blackboard model
 - Shared workspace
- **Frameworks and tool**
 - crewAI
 - Are role based agents, simple workflows and easy to use
 - Limited flexibility
 - autoGen
 - Agents talk to each other
 - Good for discussion
 - Has weak workflow control and hard to scale
 - Why langchain alone isnt enough?
 - It is designed for linear flows, it has no graph-based workflows, weak multi agent coordination, manual orchestration is required
 - Hence the need of langGraph
 - The nodes are agents/tasks
 - Edges are the connections
 - And state flows through graph
 - Features include dynamic routing, conditional logic, built in memory, debugging tools
 - Multi agent architectures in langGraph
 - Supervisor architecture
 - ◆ One master controls all agents
 - Swarm architecture
 - ◆ Agents communicate directly with each other
- **When to use and not to use multi agent systems**
 - Use when tasks are complex, need parallel processing, need error handling and workflow changes dynamically
 - Not to use when task is simple, no need for tools, need fast response
- **Challenges and consideration**
 - Overhead
 - More agents = more communication cost
 - Message design
 - Poor schema = confusion
 - Failure handling
 - Need retry logic
 - Debugging difficulty
 - Harder than single agent

- Memory consistency
 - Agents must share correct state
- Agent conflicts
 - Agents may disagree
- Complexity tradeoff

Lecture 6: From shared process memory to true multi agent systems

Tuesday, 14 April 2026 8:59 pm

From coordination to communication

- Before, agents worked together inside one system but now agents must communicate across systems
- Internal -> external communication

The problem

- Frameworks like langGraph and crewAI use shared memory
- Hidden coupling
 - All agents depend on same memory, so if one thing changes, it affects all
- Central dependency
 - Supervisor sees everything so not truly independent agents
- Single point of failure
- No privacy
 - All agents can read all data

Logical vs physical isolation

- Logical isolation
 - Agents appear separate means they have their own dict/list but share memory
 - LangGraph
- Physical isolation
 - Agents run on different machines, they communicate through network messages and have strong privacy
 - e.g. A2A or MCP

Communication is needed because

1. Microservices: different services handle different task
2. IoT devices: devices in diff location
3. Autonomous vehicles: cars communicate on road
4. Enterprise system

Understanding agent to agent (A2A) communication

- Communicate via structured messages
- Before FIPA-ACL was used, but modern A2A uses HTTP instead of old protocols and JSON instead of symbolic logic
- Each agent has a unique id, inbox, policy
- Structure includes sender, receiver, performative (type of message), content, conv_id
- How A2A works internally
 - Endpoints
 - Each agent has an address (URL)
 - HTTP like mailbox, WebSocket like live chat and MQTT like radio
 - e.g. research agent send request to analyst's endpoint
POST http://analyst.agent.net/receive_message
 - Message structure
 - JSON message is sent
 - Message routing
 - Message broker handles delivery
 - It is scalable, reliable
 - Processing
 - When message arrives, it reads message, then decides action, execute tasks and sends response

- Why A2A alone isn't enough
 - Large data transfer
 - No security
 - No provenance
 - Receiver cannot verify who sent the message, if the data is altered or if the data is real
 - No access control
- But what does A2A give?
 - Structured communication
 - Clear sender, receiver
 - Organized messaging

Communication models

- Asynchronous
 - Agents do not wait i.e. non blocking mechanism
 - Are scalable and efficient
- Synchronous
 - Sender waits for reply
- Broadcast
 - One sender and many receiver
- Peer to peer
 - Two agents interact privately

Model context protocol (MCP)

- How agents share context safely
- Its purpose is to standardize data sharing, prevent data leaks, enable verification(provenance tracking) and work across systems
- It is allowed read only access
- Core components
 - Instead of sending data, it sends pointer reference
- How MCP works
 - It stores data, generate pointer, send pointer and receiver fetches data (if allowed)
- MCP solution
 - Send pointer instead of large data e.g. send link
 - Security: has access control
 - Provenance: track who created data
 - Interoperability: works across systems

A2A + MCP

- A2A handles communication while MCP handles safe data sharing

Lecture 7: The Post-Vector Era of RAG

Wednesday, 27 May 2026 8:10 pm

- Traditional vector RAG
 - It treats documents by breaking them into random, disconnected mathematical fragments with no structure or order
- Hierarchical tree diagram
 - Root node -> tree level 1 -> tree level 2 -> data points and annotations (vectorless)
 - More organized, logical and navigable
 - Idea is to eliminate vectors entirely in favor of hierarchical trees that the LLM can reason over like a human expert

Problem #1 with traditional RAG

- Structured source document (before chunking)
 - Has clear paragraph, logical structure, a formatted data table
 - A structure, coherent, interconnected document
- Artificial chunking (Fixed token limits)
 - The document gets fed in and physically torn apart at arbitrary points, wherever the token counter reaches its limits
- Fragmented vector space (after chunking)
 - Scattered, disconnected text fragments floating in a high dimensional vector space each with a numerical embedding vector
 - Fixed size chunking blindly slices sentences, tables and logical arguments in half where an argument loses its supporting evidence
 - Context destruction where references have no connection to the actual table

Problem #2 with traditional RAG

- Even after the destruction of chunking, the retrieval method is flawed
- Semantic similarity $\neq$ logical relevance
- Illusion of proximity
 - Mathematical closeness does not equal relevance, especially in dense legal or technical domain
- Zero multi step reasoning
 - Vector RAG does a single retrieval and stops
 - Cannot adapt its search strategy based on what it found
 - Cannot ask follow up sub-questions
 - Every query is a single blind shot
- Bloated infrastructure
 - Just to retrieve documents, we need embedding model API calls, vector databases provisioning, index maintenance, storage costs and re indexing whenever documents change; enormous operational overhead

Solution : vectorless RAG

- Root node
 - Entire document is represented as a single top level node
- Branch nodes
 - Each chapter or section has its own node containing a summary of what's inside
- Leaf nodes
 - They are precise page contents
- Paradigm shift
 - Eliminates embeddings
 - Replaces dense similarity search and vector databases entirely with an LLM guided tree structure

- Human like navigation
 - LLM reads the index and navigates it step by step

Comparison : vector vs vectorless

- Retrieval method
 - Vector RAG: mathematical problem (find nearest vectors)
 - Vectorless RAG: reasoning problem (navigate to the exact place)
- Document state
 - Vector RAG: it destroys document structure
 - Vectorless RAG: preserves it entirely in a JSON tree
- Search process
 - Vector RAG: single blind shot
 - Vectorless RAG: iterative exploration (agent keeps looking until it finds what it needs)
- Infrastructure
 - Vector RAG: significant infrastructure; embedding API + vector DB
 - Vectorless RAG: just a JSON file and an LLM call

Advantage of vectorless approach

- If a document lacks an existing table of contents, the LLM reads the raw pages and infers structural headings automatically, ensuring each node covers unbroken logical idea
- Handled by PageIndex using LLM to infer structure from content
- How PageIndex stores documents for efficient navigation
 - Each node contains a summarized version of its underlying content. The LLM does not read the whole document, it reads lightweight summaries at each node- a few sentences per node and uses those summaries to navigate towards the relevant pages.
 - Only when it reaches the right leaf node does it read the full page content. This reduces token usage while increasing precision
- Retrieval loop
 - Query understanding
 - LLM receives the user's question and understands what kind of information is needed
 - Scan tree index
 - LLM reads the top level branch nodes- a few sentences each to get an overview of what the document covers at the highest level
 - Drill down
 - LLM reasons which branch is most likely to contain the answer and navigates down into it, reading the section summaries at the next level
 - Evaluate section
 - LLM reads the content of selected section and evaluates: does this actually answer the question. If no, then LLM backtracks to the parent node and tries different branch or explore adjacent sections (key advantages)
 - Generate response
 - Once the page is found, the LLM generates the answer and cites the exact node_id and page_index

Evolution toward agentic AI

- An AI agent doesn't just retrieve, it plans, decides and adapts. It chooses between web search, a document database, calculator. It re-prompts itself based on what it discovers. Even if you build the most sophisticated agent with perfect tool selection logic, if the knowledge base is a flat vector space, the agent cannot reason about what it's searching through. So give the agent a structured environment (hierarchical JSON tree)

Agents need environments, not just vectors

- Standard agent + vector RAG
 - The agent gives a query like a shotgun blast where multiple arrows are fanning out in different directions, scattering into the fragmented vector space. The result is probabilistic and chaotic and so the agent hopes one of the fragments could be relevant. There's no intelligence in the retrieval itself, pure mathematical chance it is.

- Agent + vectorless RAG
 - The agent gives a query where a single, directed arrow is sent into a structured, organized environment, represented as an interconnected grid of cubes (knowledge tree). The agent follows a clear logical path directly to the relevant node. This retrieval is purposeful, directed and navigated with intelligence

Unlocking true multi step reasoning

- 3 capabilities that vectorless RAG have
 - Iterative exploration
 - In vector RAG, when a query is sent you get back the top K results. If they're wrong you are stuck whereas in vectorless RAG it backtracks when it doesn't find a relevant summary on the current node with logical reasoning
 - Verifiable citations
 - Since every node has a node_id and page_index, every answer can be traced back to an exact, verifiable location in the original document. This near-eliminates hallucinations because the agent cannot cite a page that doesn't exist
 - Macro-synthesis
 - To summarize an entire book with vector RAG, it would have to retrieve and process of thousand of chunks but with a tree structure, the agent reads the top level branch summaries and can produce a coherent high level summary of the entire document in seconds. It make macro- level understanding easy
- Limitations of vectorless RAG
 - Dependency on structure
 - If a document is massive, poorly organized with no natural sections or hierarchy, the LLM cannot infer a good tree structure. The quality of tree determines the quality of the retrieval
 - Latency trade-off
 - Vector RAG is extremely fast as it requires one embedding call + one vector search whereas vectorless RAG requires the LLM to reason through multiple steps (scans summaries, evaluate sections, navigate the tree), which takes more time
 - Scalability profile
 - Vectorless RAG is ideal for deep, thorough analysis of structured documents. But if the use case requires searching millions of short, independent, unrelated records then tree building overhead at massive scale will be a challenge

Use cases of vectorless RAG

- Complex research paper
- Legal contracts
- Textbooks
- Highly structured corporate policies
- Technical manuals

No vector databases, no embedding model to pay. Just an SDK (page index), an api key and an LLM for reasoning